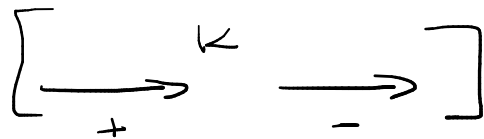


8 PUNTI... ~~TRIP OS~~ (RACCOLTA)
Domanda 21 Realizzare una funzione `Double(A,n)` che, dato un array $A[1..n]$ ordinato in senso crescente, verifica se esiste una coppia di indici i, j tali che $A[j] = 2 * A[i]$. Restituisce la coppia se esiste e (0,0) altrimenti. Scrivere lo pseudocodice e valutare la complessità.

```
double(A, n)
    i = 0, j = 0
    while (i < n) and (j < n) and (A[j] ≠ 2 * A[i])
        if (A[j] == 2 * A[i])
            return (i, j)
        else if (A[j] > 2 * A[i])
            i++
        else if (A[j] < 2 * A[i])
            j++
    return (0,0)
```



Esercizio 8 Si dica che un array $A[1..n]$ è bi-ordinato se esiste un indice k tale che $A[1..k]$ è ordinato in senso crescente e $A[k..n]$ è ordinato in senso decrescente. Realizzare un algoritmo

`top(A,n)` che dato in input un array A bi-ordinato con elementi distinti ne determina il valore massimo. Ad esempio su $A = (1\ 2\ 3\ 14\ 13\ 4)$ restituisce 14. Valutarne la complessità.

```
top(A, n)
    return top_rec(A, 1, n)

top_rec(A, p, r)
    if (p == r)
        return A[p]
    else
        q = (p + r) // 2
        if (A[q] > A[q+1])
            return top_rec(A, p, q)
        else
            return top_rec(A, q+1, r)
```

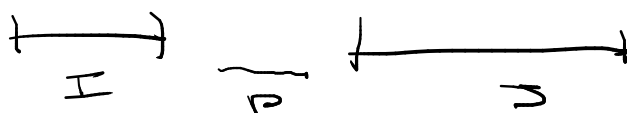
Esercizi

Esercizio 1 (7 punti) Sia $A[1..n]$ un array di interi distinti ordinato in senso crescente. Dimostrare che dato un qualunque indice i , se $A[i] > i$ allora $A[j] > j$ per ogni $j > i$ e analogamente se $A[i] < i$ allora $A[j] < j$ per ogni $j < i$.

Utilizzare l'osservazione per realizzare una funzione `Fix(A)` che dato l'array di interi $A[1..n]$ ordinato senza ripetizioni restituisce un indice i tale che $A[i]=i$, se esiste, e 0 altrimenti. Valutarne la complessità.

$$i, 3 \Rightarrow 3 > i$$

$$A[i] > i \Rightarrow A[3] > 3$$



```

Fix(A, n)
    return Fix(A, 1, n)

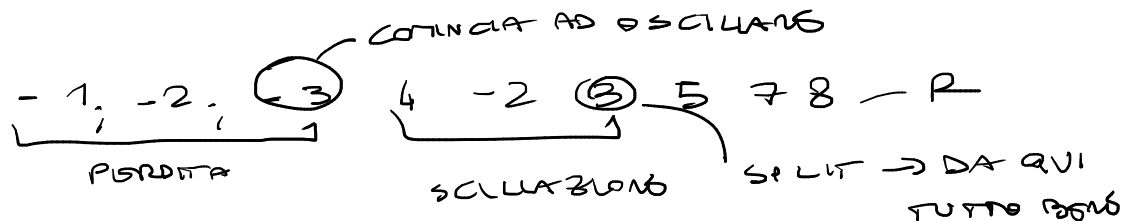
Fix_rec (A, p, r)

if p ≤ r      // i << j
    if A[q] > q
        // andando in "giù"
        return Fix_rec(A, p, q-1)
    else if A[q] < q
        return Fix_rec(A, q+1, r)

    // se i == j, return i → q
    return q
else
    return 0

```

Esercizio 1 (10 punti) Sia dato un array $V[1..n]$ i cui valori rappresentano la variazione giornaliera del valore di un titolo azionario. È noto che il titolo è stato prima perdita, con valori sempre negativi, poi ha iniziato a oscillare in giorni *consecutivi* tra valori positivi e negativi, e infine si è stabilizzato su valori positivi (dunque nella sequenza non ci possono essere due giorni positivi seguiti da un negativo). Realizzare un algoritmo *divide et impera* $\text{Split}(V)$ che individua il giorno in cui il titolo ha iniziato a essere stabile su valori positivi, ovvero il minimo indice $i \in [1, n]$ tale che per ogni $j \geq i$ vale $V[j] > 0$. Se il titolo non si stabilizza su valori positivi, ritornare 0. Ad es., se l'array $V = [-1, -2, 2, -1, 6, 3]$ l'indice da tornare sarà 5, mentre invece per $V = [-1, -2, 2, -1, 6, -3]$ si ritornerà 0. Fornire lo pseudocodice di $\text{Split}(V)$, motivarne la correttezza e individuarne la complessità. Si assuma che non ci siano valori nulli.



```

Split(V)
    n = length(V)
    return Split_rec(V, 1, n)

```

```

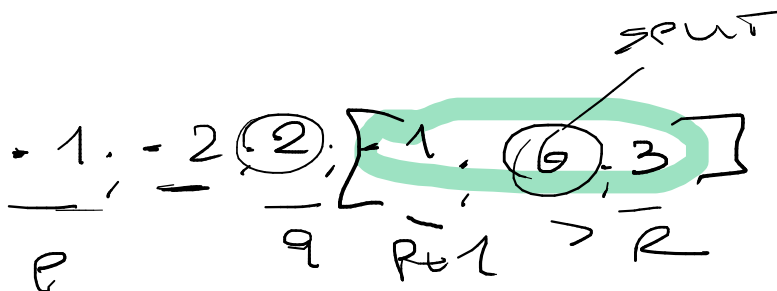
Split_rec (V, p, r)

```

```

    if (p==r)
        return V[p]
    else
        q = (p + r) // 2
        if (V[q] > 0)
            left = Split_rec(V, p, q-1)
        else
            right = Split_rec(v, q, r)
            if (V[q] > 0) and (V[q+1] < 0)
                return right
    return 0

```



```

Split_rec(V, p, r)
    if p == r
        if (V[p] > 0)
            return p
        else
            return 0
    else
        right = Split_rec(V, q+1, r)
        if (right == 0) return 0 → Non esiste stabilizzazione (contiene valori ≤ 0)
        if (right == q + 1) → Cerca stabilizzazione
            left = Split_rec(V, p, q) → Dato un positivo, trovo il primo più piccolo
            if (left > 0)
                return left // minore di q + 1 → stabile
            else
                return (q + 1) // primo indice più grande positivo → tutto right > 0
    else return right

```

Esercizio 2 (10 punti) Si consideri il problema di selezione di attività compatibili, con n attività $a_1, \dots, a_n$ che ci vengono date attraverso due vettori s e f di tempi di inizio e fine, e ordinate per tempo di inizio (cioè $0 < s_1 \leq s_2 \leq \dots \leq s_n$).

- Scrivere un algoritmo greedy iterativo che implementa la scelta greedy di selezionare l'attività che inizia per ultima.
- Determinare l'insieme di attività restituito dall'algoritmo al punto (a) quando eseguito sul seguente insieme di 6 attività, caratterizzate dai seguenti vettori s e f di tempi di inizio e fine:

$$s = (1, 2, 3, 6, 7, 9) \quad f = (4, 5, 7, 8, 12, 11)$$

- Dimostrare la proprietà di scelta greedy, cioè che esiste soluzione ottima che contiene l'attività che inizia per ultima.

① DIMOSTRA CHE LA SCELTA "ULTIMA" È OTTIMA

(a) GREEDY_SEL(s, f, n)

$A = \{a_1\}$ // similmente, a_n se parti dalla fine
 $last = 1$ // similmente $last = n$ se parti dalla fine

```
for i = 2 to n
    if (S[i] ≥ F[last])
        A = A U a_i
        last = i
```

return A

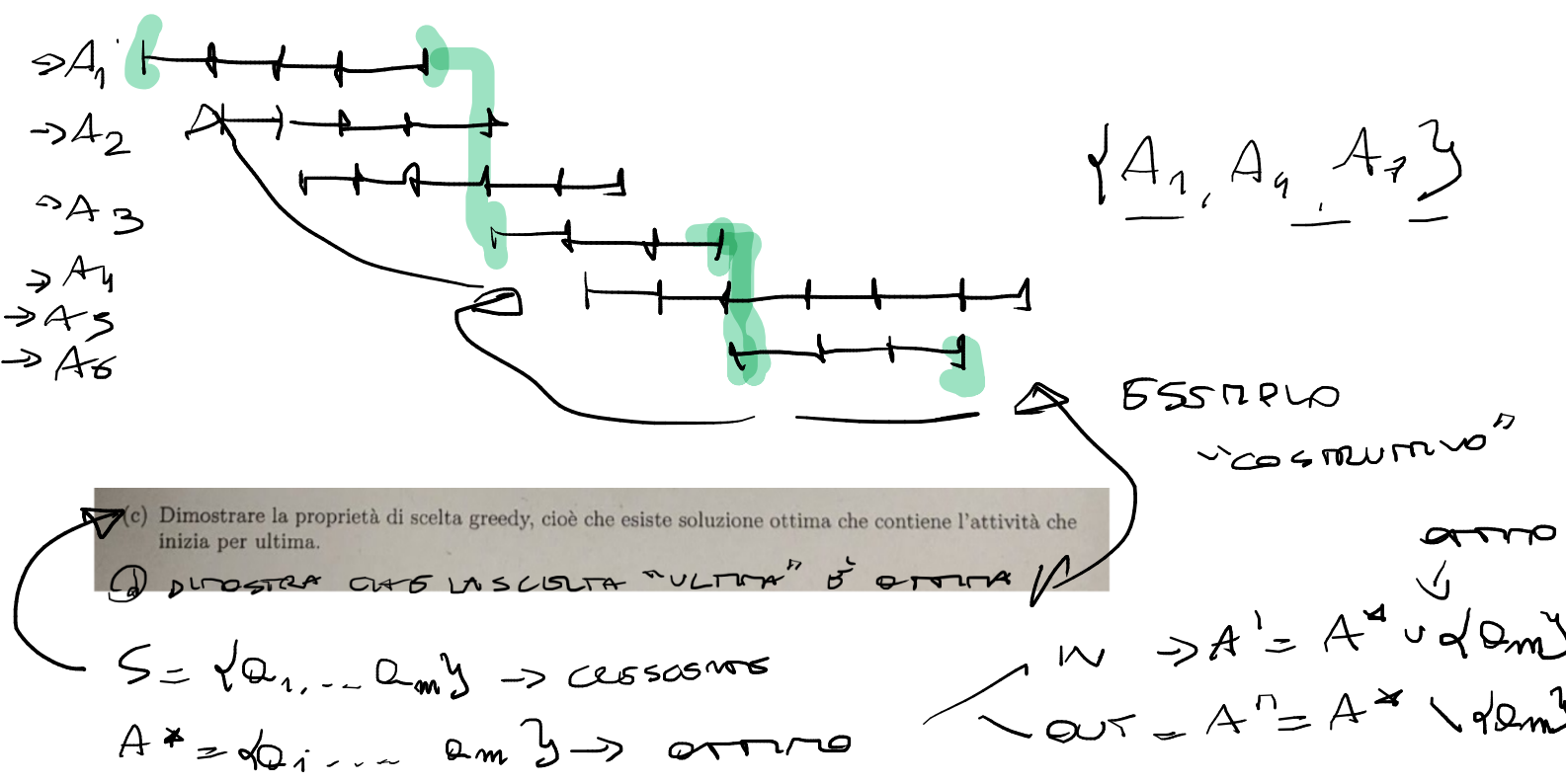
// similmente con alg. ottimo che sceglie dalla fine

```
for i = n downto 2
    if (S[i] ≤ F[last])
        A = A U a_i
        last = i
```

return A

- Determinare l'insieme di attività restituito dall'algoritmo al punto (a) quando eseguito sul seguente insieme di 6 attività, caratterizzate dai seguenti vettori s e f di tempi di inizio e fine:

$$s = (\cancel{1}, \cancel{2}, \cancel{3}, \cancel{6}, \cancel{7}, \cancel{9}) \quad f = (\cancel{4}, \cancel{5}, \cancel{7}, \cancel{8}, \cancel{12}, \cancel{11})$$



CUT-AND-PASTE $\rightarrow$ SOSTITUIRE L'ATTIVITÀ q_i

$\rightarrow$ SO "IN" $\rightarrow f_i \leq s_i$
 $\forall i \leq m$

$\rightarrow$ SO "OUT" $\rightarrow f_i \neq s_i$
 (ASSUNDO)

Esercizio 2 (8 punti) Data una stringa di numeri interi $A = (a_1, a_2, \dots, a_n)$, si consideri la seguente ricorrenza $z(i, j)$ definita per ogni coppia di valori (i, j) con $1 \leq i, j \leq n$:

$$z(i, j) = \begin{cases} a_j & \text{if } i = 1, 1 \leq j \leq n, \\ a_{n+1-i} & \text{if } j = n, 1 < i \leq n, \\ (z(i-1, j) \cdot z(i, j+1)) - z(i-1, j+1) & \text{altrimenti.} \end{cases}$$

1. Si fornisca il codice di un algoritmo iterativo bottom-up COMPUTE_Z(A) che, data in input la stringa A restituisca in uscita il valore $z(n, 1)$.
2. Si valuti il numero esatto $T_{CZ}(n)$ di operazioni tra interi eseguite dall'algoritmo sviluppato al punto (a).

COMPUTE_Z(A)

$n = \text{length}(A)$

for $i = 1$ to n

$z[1, i] = A[i]$

$i \leftarrow 1$

$z[i, n] = A[n + 1 - i]$

for $j = n - 1$ downto 1

for $i = 2$ to n

$z[i, j] = z[i-1, j] * z[i, j+1] - z[i-1, j+1]$

return $z[n, 1]$

$$\sum_{j=1}^{n-1} \sum_{i=2}^n 2 = 2 \sum_{j=1}^{n-1} (n-j) = 2(n-1)^2$$

Esercizio 2 (10 punti) Si ricordi che data una sequenza $X = x_1 \dots x_k$, si indica con X_i il prefisso $x_1 \dots x_i$. Una sottosequenza di X è $x_{i_1} \dots x_{i_h}$ con $1 \leq i_1 < i_2 < \dots < i_h \leq k$, ovvero è una sequenza ottenuta da X eliminando alcuni elementi. Quando Y è sottosequenza di X si scrive $Y \subseteq X$.

Realizzare un algoritmo che, date due sequenze $X = x_1 \dots x_k$ e $Y = y_1 \dots y_h$ determina una *shortest common supersequence* (SCS) ovvero una sequenza Z , di lunghezza minima, tale che $X \subseteq Z$ e $Y \subseteq Z$. Ad esempio per $X = abf$ e $Y = afgj$ una SCS è abfgj.

i. Dare una caratterizzazione ricorsiva della lunghezza $l_{i,j}$ di una SCS di X_i e Y_j e dedurne un algoritmo;

ii. valutare la complessità dell'algoritmo.

}

}

+ ALG.

- $L_{\{i, j\}} =$
 - (1) $\{i == 0 \parallel j == 0\} \rightarrow i + j$
 - (2) $\{i == j \ \&\& \ (i > 0, j > 0)\} \rightarrow L_{\{i-1, j-1\}} + 1$ // attacco "il prima"
 - (3) $\{i \neq j \ \&\& \ (i > 0, j > 0)\} \rightarrow \min\{L_{\{i, j-1\}}, L_{\{i-1, j\}}\} + 1$

$$l_{i,j} = \begin{cases} i+j & \text{if } i=0 \text{ or } j=0 \\ l_{i-1,j-1} + 1 & \text{if } i,j > 0 \text{ e } x_i = y_j \\ \min\{l_{i,j-1}, l_{i-1,j}\} + 1 & \text{if } i,j > 0 \text{ e } x_i \neq y_j \end{cases}$$

$\rightarrow \text{NOT}$
 $\rightarrow \text{RISPARMIARE}$

Ne segue l'algoritmo che riceve in input le stringhe, nella forma di array di caratteri $X[1..k]$, $Y[1..h]$ e usa una matrice $L[0..k, 0..h]$ dove $L[i, j]$ rappresenta la lunghezza della minima SCS di X_i e Y_j .

```

SCS_len(X, Y, k, h)
    for i = 1 to k
        for j = 2 to h
            if (X[i] == Y[j])
                L[i, j] = L[i-1, j-1] + 1
            else
                L[i, j] = min( ... ) + 1
    return L[k, h] → m, n
  
```

if (i == 0) || (j == 0) return 0
 for i = 0 to k
 L[i, 0] = i
 for j = 1 to h
 L[0, j] = j

$\begin{matrix} > > i \\ \wedge > 0 \end{matrix}$

Esercizio 2 (11 punti) Una *longest common substring* di due stringhe X e Y è una sottostringa di X e di Y di lunghezza massima. Si vuole progettare un algoritmo efficiente per calcolare la lunghezza di una longest common substring. Per semplicità si assuma che entrambe le stringhe di input abbiano stessa lunghezza n .

BRUTEFORCE

- (a) Qual è la complessità dell'algoritmo eshaustivo che analizza tutte le possibili sottostringhe comuni?
- (b) Assumendo di conoscere un algoritmo che determina se una stringa di m caratteri è sottostringa di un'altra stringa di n caratteri in tempo $O(m+n)$, come si può modificare l'algoritmo del punto precedente per renderlo più efficiente?
- (c) Progettare un algoritmo di programmazione dinamica più efficiente di quello del punto precedente. Sono richiesti relazione di ricorrenza sulle lunghezze (senza dimostrazione) e algoritmo bottom-up. (Suggerimento: considerare la lunghezza della longest common substring dei prefissi $X_i = \langle x_1, \dots, x_i \rangle$ e $Y_j = \langle y_1, \dots, y_j \rangle$ che termina con x_i e y_j , rispettivamente.)

$X, Y \quad m, n \Rightarrow \text{STESSA LENGTH}$

Ⓐ **ESHAUSTIVO?** $1 \leq i \leq n \leq m \quad \Theta(n^2) = \frac{n(m+n)}{2}$

Ⓑ $m \leq n$ (SOTTOSTRINGA → SOTTOSTRINGA)
 $\Theta(n^2) \Rightarrow \Theta(m+n) \Rightarrow \text{ALGORITMO?}$

FOR $i = 1$ TO N
 FOR $j = 1$ TO N

$\downarrow$ $n^2 \rightarrow m[i, j] = m[i, j-1]$
 $m = m[i+1, j]$
 $(m+n) \rightarrow m[m+n] + \dots + m$


```

COMPUTE(a,b)
n <- length(a)
M = -infinito
for i=1 to n-1 do
    C[i,n-1] <- a_i
    M <- MAX(M, C[i,n-1])
for j=0 to n-1 do
    C[0,j] <- b_j
    M <- MAX(M, C[0,j])
for i=1 to n-2 do
    for j=n-2 downto i do
        C[i,j] <- C[i-1,j] * C[i,j+1]
        M <- MAX(M, C[i,j])
return M

```

LCS(X,Y)

```

1 m = X.length
2 n = Y.length
3 for i = 0 to m
4     L[i,0] = 0
5 for j = 0 to n
6     L[0,j] = 0
7 for i = 1 to m
8     for j = 1 to n
9         if x_i = y_j
10            L[i,j] = L[i-1,j-1] + 1
11            B[i,j] = '↖'
12        else if L[i-1,j] >= L[i,j-1]
13            L[i,j] = L[i-1,j]
14            B[i,j] = '↑'
15        else
16            L[i,j] = L[i,j-1]
17            B[i,j] = '←'
18 return (L[m,n], B)

```

EXHAUSTIVE_LCS_WITH_ORACLE(X, Y, n)

```

max_length = 0
lcs = ""

```

```

for i = 1 to n
    for j = i to n
        substring = X[i..j]
        k = j - i + 1

```

```

// inizio sottostringa
// fine sottostringa
// estrai sottostringa
// lunghezza sottostringa

```

```

if IS_SUBSTRING(substring, Y, k, n) // O(k + n)
    if k > max_length
        max_length = k
        lcs = substring

```

return lcs

Esercizio 2 (11 punti) Una *longest common substring* di due stringhe X e Y è una sottostringa di X e di Y di lunghezza massima. Si vuole progettare un algoritmo efficiente per calcolare la lunghezza di una longest common substring. Per semplicità si assuma che entrambe le stringhe di input abbiano stessa lunghezza n .

- Qual è la complessità dell'algoritmo esaustivo che analizza tutte le possibili sottostringhe comuni?
- Assumendo di conoscere un algoritmo che determina se una stringa di m caratteri è sottostringa di un'altra stringa di n caratteri in tempo $O(m + n)$, come si può modificare l'algoritmo del punto precedente per renderlo più efficiente?
- Progettare un algoritmo di programmazione dinamica più efficiente di quello del punto precedente. Sono richiesti relazione di ricorrenza sulle lunghezze (senza dimostrazione) e algoritmo bottom-up. (Suggerimento: considerare la lunghezza della longest common substring dei prefissi $X_i = \langle x_1, \dots, x_i \rangle$ e $Y_j = \langle y_1, \dots, y_j \rangle$ che termina con x_i e y_j , rispettivamente.)

LCS(X,Y)

```

1 m = X.length
2 n = Y.length
3 for i = 0 to m
4     L[i,0] = 0
5 for j = 0 to n
6     L[0,j] = 0
7 for i = 1 to m
8     for j = 1 to n
9         if x_i = y_j
10            L[i,j] = L[i-1,j-1] + 1
11            B[i,j] = '↖'
12        else if L[i-1,j] >= L[i,j-1]
13            L[i,j] = L[i-1,j]
14            B[i,j] = '↑'
15        else
16            L[i,j] = L[i,j-1]
17            B[i,j] = '←'
18 return (L[m,n], B)

```

$L[i,s] = \begin{cases} 0 & i=0, s=0 \\ L[i-1,s-1] & x_i = x_s \\ \max\{L[i-1,s], L[i,s-1]\} & x_i \neq x_s \end{cases}$

$s = m-1$ down to 1

$m = \max\{m, \dots\}$

$L[i-1,s-1] + 1$

(UGUALS ROLGU ALTRA...)

$$l(i, j) = |LCS(X_i, Y_j)|$$

$$l(i, j) = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \quad (\text{caso 0}) \\ l(i-1, j-1) + 1 & \text{se } i, j > 0 \text{ e } x_i = x_j \quad (\text{caso 1}) \\ \max\{l(i, j-1), l(i-1, j)\} & \text{se } i, j > 0 \text{ e } x_i \neq x_j \quad (\text{caso 2}) \end{cases}$$

Alla fine ci interessa calcolare $l(m, n)$.

Per calcolare $l(i, j)$ mi possono servire tre valori:

$$L = \begin{bmatrix} & (i-1, j-1) & (i-1, j) \\ & \swarrow & \uparrow \\ (i, j-1) & \leftarrow & (i, j) \end{bmatrix}$$

INIT-LCS(X, Y)

```

1  m = X.length
2  n = Y.length
3  if (m = 0) or (n = 0)
4    return 0
5  for i = 0 to m
6    L[i, 0] = 0
7  for j = 0 to n
8    L[0, j] = 0
9  for i = 1 to m
10   for j = 1 to n
11     L[i, j] = -1
12  return R-LCS(X, Y, m, n)

```

R-LCS(X, Y, i, j)

```

1  if L[i, j] = -1
2    if x_i = y_j
3      L[i, j] = R-LCS(X, Y, i-1, j-1)
4    else if R-LCS(X, Y, i-1, j) > R-LCS(X, Y, i, j-1)
5      L[i, j] = L[i-1, j]
6    else
7      L[i, j] = L[i, j-1]
8  return L[i, j]

```

LCS(X, Y)

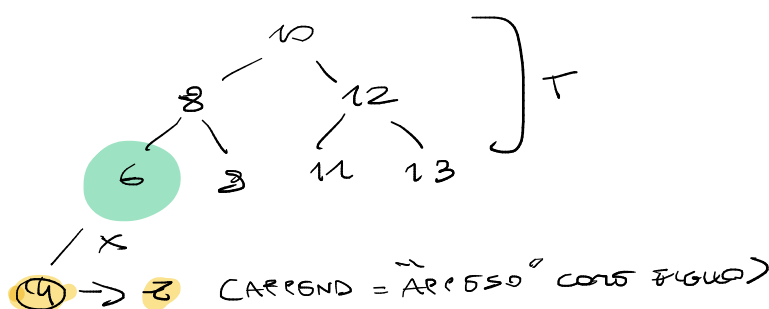
```

1  m = X.length
2  n = Y.length
3  for i = 0 to m
4    L[i, 0] = 0
5  for j = 0 to n
6    L[0, j] = 0
7  for i = 1 to m
8    for j = 1 to n
9      if x_i = y_j
10       L[i, j] = L[i-1, j-1] + 1
11       B[i, j] = '↖'
12     else if L[i-1, j] > L[i, j-1]
13       L[i, j] = L[i-1, j]
14       B[i, j] = '↑'
15     else
16       L[i, j] = L[i, j-1]
17       B[i, j] = '←'
18  return (L[m, n], B)

```

(SOPRA
Bottom-up)
TOP-DOWN
(LCS)
VALORIZZAZIONE

Esercizio 1 (9 punti) Realizzare una funzione $append(T, x, z)$ che dato un albero binario di ricerca T , un suo nodo foglia x ed un nuovo nodo z , verifica se è possibile inserire z come figlio (destro o sinistro) di x in T . In caso affermativo ritorna l'albero esteso, altrimenti ritorna nil. (Assumere che i nodi dell'albero abbiano gli usuali campi $x.left$, $x.right$, $x.p$, $x.key$ e la radice sia $T.root$.) Valutarne la complessità.



append(T, x, z)

```

# verifica che z.key sia <= delle chiavi degli antenati di x
# per i quali x e' nel sottoalbero dx e
y = x

```

```

while (y.p <> nil) and
  ( (y == y.p.left and z.key <= y.p.key) or
    (y == y.p.right and z.key >= y.p.key) )
  y = y.p

```

```

# se siamo arrivati alla radice, significa che la condizione e' soddisfatta
if (y.p == nil)
  # e z viene inserito come figlio dx o sx
  z.p = x
  if (z.key <= x.key)
    x.left = z
  else
    x.right = z

```

return T

```

else
  return nil

```

append(T, x, z)

```

if (x == nil) return nil

```

```

// logica "simil" insert

```

```

y = x

```

```

while (x != nil)
  if(z.key <= x.key)
    x = x.left
  else
    x = x.right

```

```

z.p = y

```

```

while(y != nil)
  z.p = x
  if(z.key <= x.key)
    x.left = z
  else
    x.right = z

```

```

return T

```

Soluzione: Affinchè z possa essere inserito come figlio destro o sinistro di x , occorre in primo luogo che la chiave di z sia $\leq$ delle chiavi degli antenati di z che hanno x e quindi z nel sottoalbero sinistro e $\geq$ delle chiavi degli antenati di x che hanno x nel sottoalbero destro.

Quindi risalendo tali antenati, si verifica che la condizione sia soddisfatta. Se questo accade e $z.key \leq x.key$ potrà essere inserito come figlio sinistro di x , altrimenti come figlio destro. Altrimenti z non può essere figlio di x e si ritorna nil.

Domanda A (8 punti) Si consideri la seguente funzione ricorsiva con argomento un intero $n \geq 0$

```
exp(n)
  if n = 0
    return 0
  else
    return exp(n-1) + exp(n-1) + 1
```

Dimostrare induttivamente che la funzione calcola il valore $2^n - 1$. Inoltre, determinare la ricorrenza che esprime la complessità della funzione e mostrare che la soluzione è $\Omega(2^n)$. È anche $O(2^n)$? Motivare le risposte.

$$T(n) = \underline{2T(n-1) + c}$$

$$\left\{ T(n) = \Omega(n) \rightarrow \exists d > 0, n_0 \in \mathbb{N} \mid T(n) \geq d \cdot 2^n \right. \\ \left. \forall n \geq n_0 \right\}$$

$$2T(n+1) + c \quad \frac{2d \cdot 2^{n+1} - 1}{2^n} \geq \underline{d \cdot 2^n} \quad (c > 0) \\ \Rightarrow$$

$$2 \cdot \frac{1}{2} \quad d \geq 0, \forall n \geq 0$$

$$O(2^n) \rightarrow d > 0, \exists n_0 \in \mathbb{N}, T(n) \leq d \cdot 2^n$$

$$T(n) = 2T(n-1) + c \quad \text{MESSO DA L'IPOTESI RICORRENZA}$$

$$\Rightarrow 2 \cdot d(2^{n-1} - 1) + c \leq d \cdot 2^n$$

$$\underline{d \cdot 2^n - 1} \leq d \cdot 2^n \quad \xrightarrow{\text{H.P. INDUTT.}} \text{FALSO}$$

Domanda 8 Sia data la seguente equazione di ricorrenza:

$$T(n) = T(n-1) + \log n$$

Si dimostri che $T(n) = \underline{O(n \log n)}$.

Soluzione: Si procede una prova induttiva del fatto che $T(n) \leq cn \log n$.

$$\begin{aligned} T(n) &= T(n-1) + \log n \\ &\leq \underline{c(n-1) \log(n-1)} + \log n \\ &\leq c(n-1) \log n + \log n \\ &= (cn - c + 1) \log n \\ &\leq cn \log n \end{aligned}$$

$$\begin{aligned} &C(n \log(n)) \\ &\quad \downarrow \quad \downarrow \\ &n-1 \quad n-1 \end{aligned}$$

per $c \geq 1$.

$$T(n) \leq d(2^n)$$

$$\hookrightarrow T(n) \leq d(2^{n-1})$$

INDUZIONE

$$T(n) \leq F(n)$$

$$\hookrightarrow T(n) \leq F(n-1)$$